

The CESK Machine and Its Stepper in Block-MiniJava

Machine Semantics, UI Architecture, and Source Mapping

Research note — Block-MiniJava

2026-07-20

Abstract

Block-MiniJava’s ”CESK” viz-dock tab steps a real, faithful MiniJava program one machine transition at a time and renders its configuration as four coordinated panels — Control, Stack+Environment, Store, Kontinuation — with SVG arrows tracing every reference into the heap box it points at. This note explains how that machine and its stepper actually work, at two levels: the *semantics* (what a CESK machine is, and which configuration this project’s machine realizes), and the *implementation* (exactly which TypeScript type, function, and line implements which theoretical component, and how the interactive stepper in `stepperPanel.ts` turns a pure transition function into a time-travelling, provenance-linked visualization). A companion tab, ”A vs B” (`comparePanel.ts`), reuses the identical machine to run two value models in lockstep, and is covered briefly in §5 as a one-page consequence of the same design.

Contents

1	What “the CESK machine” means here	2
2	The machine, formally	2
2.1	Configuration	2
2.2	The transition function	2
2.3	Purity, and why it matters for the UI	3
2.4	Rule summary	3
3	Theory-to-implementation mapping	3
3.1	The kontinuation vocabulary	4
3.2	Effects: the visualization hook	5
4	The stepper: from a pure function to an interactive, provenance-linked UI	5
4.1	Module state and the control loop	5
4.2	Rendering: one function per CESK column	6
4.3	Provenance: everything links back to a block	7
4.4	Value chips and the heap-arrow visualization	7
4.5	HTML wiring	7
5	The sibling stepper: A vs B lockstep	8
6	Verification	8
7	Theoretical sources	9
	Note on scope	9

1 What “the CESK machine” means here

Block-MiniJava’s Model A evaluator (`src/core/semantics/minijavaMachine.ts`) is a **CESK machine** — Control, Environment, Store, Kontinuation — in the sense of Felleisen and Friedman’s CEK machine [1] extended with an explicit, imperative-language store as formalized by Might and Van Horn [2], further extended with an explicit *stack of activation frames* (rather than one shared continuation) to support method call/return, tracing back to the *dump* of Landin’s SECD machine [3]. This is the same machine formalized in the companion note `typing-and-evaluation-rules.tex` (§4.2); *this* note’s job is different: go deep on *how the machine and its UI are actually built*, with every theoretical component pinned to a file, a type, and a function.

Three things distinguish the CESK tab from the language’s other two steppers, and are worth having side by side before diving in:

	CESK (this note)	A vs B / Rewrite
value model	Model A only (heap, Ref)	A&B lockstep / B only
state shape	one <code>MachineState</code>	a pair, or a substitution tree
UI file	<code>stepperPanel.ts</code>	<code>comparePanel.ts</code> / (Rewrite panel)
unique feature	heap boxes + SVG arrows + GC	side-by-side divergence / redex highlight

2 The machine, formally

2.1 Configuration

A machine state is

$$\Sigma = \langle C, \varphi:\Phi, H \rangle, \quad \varphi = \langle \text{self}, \rho, \kappa \rangle$$

with C the *control* (a statement, an expression, a produced value, or done), $\varphi:\Phi$ a non-empty stack of activation frames (top frame φ , the rest Φ), and H the heap. Each frame is itself a CEK-style triple: **self** (the frame’s **this**, a heap location under Model A), ρ (its local environment: parameters and locals, held *by copy*), and κ (its own pending-work stack, innermost first). **Every frame owns its own κ** — this is what makes the design a *stack of CEK configurations* rather than one flat CESK machine with a single global continuation: a paused caller frame keeps exactly the pending work it had when it made the call, and control returns to precisely that point when the callee’s frame pops.

2.2 The transition function

Execution is a (partial) function $\text{step} : \Sigma \rightarrow \Sigma$, total on every Σ with `status = running`, and it dispatches on the shape of C :

$$\text{step}(\Sigma) = \begin{cases} \text{beginStatement}(\Sigma, s) & C = \text{Stmt}(s) \\ \text{beginExpression}(\Sigma, e) & C = \text{Expr}(e) \\ \text{applyKont}(\Sigma, v) & C = \text{Value}(v) \\ \text{applyKont}(\Sigma, \perp) & C = \text{Done} \end{cases}$$

`beginStatement`/`beginExpression` *decompose* a statement or expression — pushing whatever continuation records “what to do with the result” and refocusing C onto the first subterm that needs evaluating (or, for a value-producing leaf like a literal, going straight to `Value(v)`). `applyKont` is the dual: given a value (or $\perp$, meaning “this statement finished with no value”), it pops one continuation frame off κ and does whatever that frame was waiting to do — fold an operator, write a binding, dispatch an **if**, push or pop a call frame, and so on. This decompose/recompose discipline is exactly how a CEK machine implements evaluation-context congruence *without*

representing the context as data: the continuation stack *is* the context, built up one push at a time as the machine walks down into a term and unwound one pop at a time as it walks back up with a value.

2.3 Purity, and why it matters for the UI

step is a *pure* function: given Σ , it returns a *fresh* Σ' that shares no mutable substructure with Σ (frames, locals maps, kont stacks, and the heap are all shallow-copied at the top of every step). This single property is what makes the stepper’s **Back** button exact time travel rather than an undo log that has to know how to reverse each rule: the UI simply keeps every previously visited Σ on a plain array and pops the last one to go back (§4.1). No rule in §2.4 needs an inverse.

2.4 Rule summary

The full inference-rule presentation of every named transition (LIT, VAR, FIELD-WRITE vs. FIELD-UPDATE, CALL, RETURN, and every operator/array/string rule) is in the companion note `typing-and-evaluation-rules.tex`, §4.2.3 — this note does not repeat it, and instead spends its space on §3 and §4, which that note does not cover.

3 Theory-to-implementation mapping

Every symbol in §2 corresponds to one, and only one, place in `src/core/semantics/minijavaMachine.ts`:

Theory	TypeScript	Notes
Σ (state)	<code>MachineState</code> (l.125–142)	<code>{model, control, stack, heap, output, nextLoc, status, error, table, stepCount, lastRule, lastEffect, focusBlockId}</code> — the last three fields exist purely for the visualization (§4), not for the semantics.
C (control)	<code>Control</code> (l.108–112)	A tagged union: <code>{tag: 'Stmt', block}, {tag: 'Expr', block}, {tag: 'Value', value}, {tag: 'Done'}</code> . <code>block</code> is a live <code>Blockly.Block</code> reference — the machine’s “syntax” is the program’s own block tree, read but never copied.
Φ (frame stack)	<code>MachineState.stack: Frame[]</code> (l.128)	Pushed by <code>pushFrame</code> (l.298–358) on a method call, popped inside <code>applyKont</code> ’s empty-continuation case (l.688–693) on return.
φ (one frame)	<code>Frame</code> (l.88–106)	<code>{method, self, selfObj, locals, kont, blockId, callBlockId, returnBlock, returningNow}</code> . <code>self</code> is Model A’s <code>this</code> (a <code>Loc</code> or <code>null</code> in <code>main</code>); <code>selfObj</code> is Model B’s (always <code>null</code> here).
ρ (environment)	<code>Frame.locals: Map<string, MachineValue></code> (l.96)	Parameters and locals, by copy — binding a <code>Ref</code> copies the arrow (four bytes, conceptually), not the heap object it points to.
κ (kontinuation)	<code>Frame.kont: Kont[]</code> (l.97), <code>Kont</code> union (l.64–86)	Nineteen tags across three groups (state-ment, operator, method-call continuations) — §3.1.

Theory	TypeScript	Notes
H (store)	<code>MachineState.heap:</code> <code>Map<Loc, HeapObj></code> (1.130)	<code>HeapObj</code> (1.57–59) is <code>{tag: 'Obj', className, fields, blockId}</code> or <code>{tag: 'Arr', elems, blockId}</code> — every heap object also carries the <code>new/new int[]</code> block that allocated it, purely for provenance (§4).
v (values)	<code>MachineValue</code> (1.46–53)	<code>Int Bool Str Null Ref(loc)</code> (Model A) plus <code>Obj Arr</code> (Model B only, unused on this tab).
<code>step</code>	<code>step(previous)</code> (1.936–951)	<code>cloneState</code> (1.182–200) first (the purity in §2), then dispatches on <code>state.control.tag</code> to one of the four handlers below.
<code>beginStatement</code>	<code>beginStatement(state, block)</code> (1.393–433)	One case per statement block type (<code>mj_statement_block/if/while/print/assign/array_assign</code>); pushes the matching <code>Kont</code> and refocuses <code>C</code> on the first subexpression, or recurses into a nested statement chain via <code>enterChain</code> (1.237–244).
<code>beginExpression</code>	<code>beginExpression(state, block)</code> (1.435–571)	One case per expression block type; literals go straight to <code>produce</code> (1.228–230, sets <code>Value</code>); everything else pushes a <code>Kont</code> and refocuses on a child.
<code>applyKont</code>	<code>applyKont(state, value)</code> (1.667–933)	Pops one <code>Kont</code> , or — if κ is empty — handles frame exhaustion: focus the method's <code>return</code> expression (<code>RETURN-FOCUS</code>), deliver a value back to the caller and pop the frame (<code>RETURN</code>), or halt (<code>HALT</code>) once the outermost (<code>main</code>) frame is done.
initial Σ	<code>injectMachine(workspace, model)</code> (1.954–1003)	Builds the one-frame <code>main</code> stack from the workspace's <code>mj_goal/mj_main_class</code> blocks; returns <code>{injectError: string}</code> instead when the program is structurally incomplete (no <code>goal/main-class</code> block) rather than throwing.
run-to-completion	<code>run(initial, maxSteps)</code> (1.1006–1013)	Iterates <code>step</code> under a step-count bound; used by the non-interactive test suites, <i>not</i> by the interactive stepper (which calls <code>step</code> once per user action instead, §4.1).

3.1 The kontinuation vocabulary

Group	Tags	Meaning (what fires when the hole is filled)
statements	KStmtSeq, KLoop, KIf, KWhile, KPrint, KAssign, KArrAssignIdx, KArrAssignVal	Sequencing (advance to the next statement, or finish the block), loop re-entry, branch dispatch, output, and the two-stage index-then-value continuation for <code>a[i]=e</code> .
operators	KNot, KAnd, KOr, KBinL, KBinR	KAnd/KOr implement short-circuit directly (the right operand's block is only ever focused if the left value does not already decide the result); KBinL→KBinR sequences left-then-right for every strict binary operator (arith, compare, <code>concat</code>).
arrays/strings	KLookupArr→KLookupIdx, KLength, KNewArr, KCharAtStr→KCharAtIdx, KStrLength	Each two-stage pair evaluates its receiver/array first, then the index, mirroring the left-to-right evaluation order of the surface syntax.
method call	KCallRecv→KCallArgs	Receiver first, then arguments strictly left-to-right (KCallArgs.done accumulates already-evaluated arguments, .remaining the still-unevaluated ones); once the last argument lands, <code>pushFrame</code> (l.298) fires — dynamic dispatch resolved from the receiver's <i>runtime</i> class, not its static type.

3.2 Effects: the visualization hook

`MachineState.lastRule/lastEffect` are not read by `step` itself — no rule branches on them — they exist solely so the stepper can explain, in the UI, *what just happened*. `Effect` (l.115–123) is a small closed set — `new`, `field-write`, `arr-write`, `local-write`, `self-write`, `push-frame`, `pop-frame`, `output` — and it is this tag, not the rule name, that `stepperPanel.ts`'s rendering keys off to decide which heap box or locals row to flash (§4).

4 The stepper: from a pure function to an interactive, provenance-linked UI

`src/core/ui/stepperPanel.ts` (876 lines) is everything between "a pure `step` function exists" and "a user can press *Step*, see four coordinated panels update, and click any value to jump to the block that produced it." Its job splits into five concerns.

4.1 Module state and the control loop

Symbol	Role
<code>current:</code> <code>MachineState null</code> (1.44)	The machine state currently on screen. <code>null</code> before the first Load .
<code>history:</code> <code>MachineState[]</code> (1.45)	Every previously visited state, oldest first, capped at <code>MAX_HISTORY = 5000</code> (1.31). Back is exactly <code>current = history.pop()</code> (<code>stepBack</code> , 1.822–828) — possible <i>only</i> because <code>step</code> is pure (§2); nothing is ever un-mutated.
<code>stale: boolean</code> (1.46)	Set the instant the workspace changes while a machine is loaded (<code>markStale</code> , 1.143–150, driven by a Blockly <code>addChangeListener</code> in <code>attachWorkspaceListener</code> , 1.152–173) — the stepper never silently keeps stepping a program that no longer matches what is on screen; every control disables until the user presses Load again.
<code>loadMachine()</code> (1.775–797)	Load: calls <code>injectMachine</code> , resets <code>history</code> and <code>stale</code> , and (re)installs the workspace-change listener.
<code>stepOnce()</code> (1.799–820)	Step: pushes <code>current</code> onto <code>history</code> , then <code>current = step(current)</code> ; also checks the GC auto-trigger (companion note <code>garbage-collection.tex</code> , §4.3) after every mutator step.
<code>togglePlay()</code> (1.830–839)	Play/Pause: a <code>setInterval</code> calling <code>stepOnce</code> every <code>PLAY_INTERVAL_MS = 550ms</code> (1.30) — auto-stops on <code>status</code> leaving <code>'running'</code> , on <code>stale</code> , or while a GC animation owns the machine.
<code>initStepperPanel(getWorkspace)</code> (1.857–876)	The module’s one exported wiring entry point — called once from <code>app.ts:1413</code> with a closure returning the live <code>Blockly.WorkspaceSvg</code> . Attaches every button/checkbox listener, a scroll listener per panel body and a <code>ResizeObserver</code> (both just to keep the SVG arrows aligned, §4.4), and does the first <code>renderAll()</code> .

4.2 Rendering: one function per CESK column

`renderAll()` (1.480–490) is the single entry point every state change routes through (`loadMachine`, `stepOnce`, `stepBack`, and the GC driver all end by calling it, directly or via `finishSweep`). It calls, in order, `renderStatus`, `renderButtons`, then one function per visual column:

Column	Function	What it shows
Control	<code>renderControl</code> (1.225–271)	A “kind” chip (statement/expression/value/done), the focused block’s text (click-to-locate via <code>locateProvenance</code> , 1.90–100) or the produced value as a chip, and the <code>lastRule</code> name — the one place in the UI the rule names from §2.4 are printed verbatim.
S·E	<code>renderFrames</code> (1.367–411)	The call stack, top frame first, each as a card: the method name (click-to-locate the call site), <code>this</code> → a <code>Ref</code> chip when applicable, and a table of <code>locals</code> — a row flashes <code>is-changed</code> exactly when <code>lastEffect.kind===‘local-write’</code> named that local, on the top frame only.

Column	Function	What it shows
Store	<code>renderHeap</code> (1.413–462)	One box per live heap object/array, sorted by address; a box flashes is-changed on a fresh 'new' at that location, or is-write-target on a 'field-write' / 'arr-write' there; the changed field/element row inside gets is-changed too. Every box's title is click-to-locate to the new block that allocated it (<code>HeapObj.blockId</code>).
Kontinuation	<code>renderKont</code> (1.315–365)	Every live frame's own κ , innermost entry first, rendered via <code>kontEntry</code> (1.284–313) — a per-tag mapping from a <code>Kont</code> value to a human-readable "hole" sentence, e.g. <code>KAssign</code> → " <code>x = □</code> ", <code>KBinR</code> → " <code><left-chip> + □</code> ". The next entry to fire on the top frame is marked is-next .
Output	<code>renderOutput</code> (1.464–478)	The accumulated <code>println</code> log, flashing on <code>lastEffect.kind==='output'</code> , mirrored into the shared bottom <i>Output</i> tab via <code>mirrorProgramOutput</code> so Run and the CESK stepper share one console view.

4.3 Provenance: everything links back to a block

Every rendered artifact — the focused control, a frame's name, a heap box's title, a kontinuation entry with a waiting block — is a clickable `.stepper-provenance` element wired to `locateProvenance(blockId)` (1.90–100), which selects and centers that block in the *main workspace* (not a separate mini-view): the CESK panel never draws its own copy of the program, it always points back at the one live program surface. `setHighlight` (1.119–133) additionally keeps the machine's current `focusBlockId` highlighted continuously, so the program surface itself shows where execution currently is even without a click.

4.4 Value chips and the heap-arrow visualization

Every `MachineValue` is rendered by `valueChip` (1.72–86): scalars/null print their value directly; a `Ref(loc)` becomes a chip colored by `locHue(loc) = (loc*67) mod 360` (1.68–70) — "the poor man's arrow: two chips with one color are two arrows to one box" (module docstring, 1.9–10). The literal arrows are drawn separately, on demand, by `drawArrows` (1.524–~600) directly into an overlay `<svg id="stepper-arrows">` sized to the whole panel group: every element in the DOM carrying a `data-ref-loc` attribute (a chip anywhere in Control, Frames, Heap, or Kont) gets a cubic-Bézier path drawn from its position to its target heap box, using the same `--loc-hue` custom property for stroke color as the chip itself, so a color match *is* the visual proof of aliasing. On a `field-write/arr-write` step the written box's incoming arrow additionally gets an animated dot (an SVG `<animateMotion>`) travelling from the writing frame's chip into the box — the "money animation" the design note calls out (`block-minijava-value-and- visualisation-design.md`, §6): the value's motion is drawn along the *same* arrow that was already there, never a new one, which is the visual argument that a field write mutates a shared cell rather than creating a new one. Redraws are batched to one per animation frame (`scheduleArrowRedraw`, 1.503–512) and re-triggered on panel scroll/resize so the arrows never desync from element positions the user just moved by scrolling.

4.5 HTML wiring

Element	Location
<code><button class="viz-tab" data-kind="machine"></code>	<code>src/index.html:260--262</code> — the "CESK" tab button in the viz-dock's tab strip; its title attribute is the one-sentence description ("Control · Environment · Store · Kontinuation, over an activation-frame stack") shown as a tooltip.
<code><div data-kind="machine"></code>	<code>index.html:287--327</code> — the panel body: the control strip (stepper-load/back/step/play/gc buttons, the GC threshold inputs, and <code>stepper-status</code>) above the four-column <code>.stepper-panels-csesk</code> grid (<code>stepper-control</code> , <code>stepper-frames</code> , <code>stepper-heap</code> , <code>stepper-kont</code> , plus an Output card) and the <code><svg id="stepper-arrows"></code> overlay.
<code>initStepperPanel() => workspace)</code>	<code>app.ts:1413</code> — the one call site that wires this whole module to the live IDE workspace; <code>app.ts:364</code> additionally calls <code>scheduleTypingRender</code> -style visibility hooks so Play stops when the tab is hidden (<code>setStepperTabVisible</code> , 1.847–855).

5 The sibling stepper: A vs B lockstep

`src/core/ui/comparePanel.ts` is a second, much smaller UI (reusing `valueChip/locHue/step` against the *same* `miniJavaMachine.ts`) that demonstrates the design note's §8 artifact directly: one program, two machines.

Symbol	Role
<code>stateA, stateB</code>	Two independent <code>MachineStates</code> , both built by <code>loadBoth()</code> (1.290–312) calling <code>injectMachine(workspace, 'A')</code> and <code>injectMachine(workspace, 'B')</code> on the <i>same</i> workspace — the only difference between them is the <code>model</code> tag threaded through every subsequent <code>step</code> call.
<code>stepBoth()</code> (1.314–326)	Advances <i>both</i> machines by exactly one transition each, <code>stateA = step(stateA); stateB = step(stateB)</code> — because both share the same control-flow skeleton (§2), their <code>stepCounts</code> stay equal by construction; they diverge only in <i>values</i> (a shared cell in A, an independent copy in B), which is exactly the pedagogical point.
<code>history: Array< MachineState, MachineState]></code>	A history of <i>pairs</i> , so Back undoes one lockstep step on both machines at once — the same purity-enables-time-travel argument as §4.1, applied twice.

No new machine semantics are introduced here — `comparePanel.ts` is UI-only evidence that parameterizing `step` by `model` (rather than writing two separate interpreters) was the right factorization: running Model A and Model B "in lockstep" is not a special mode, it is simply calling the same function twice with a different tag.

6 Verification

`test/run_machine.js` (part of `npm test`'s five headless suites) runs each of many MiniJava programs through `text` $\rightarrow$ `blocks` $\rightarrow$ `injectMachine` $\rightarrow$ `step*` and asserts on the full run report: printed output, final `status`, error-message fragments for stuck programs, which rules were exercised, maximum call-stack depth, and final heap size — so every transition rule in §2.4 is exercised by name, not just indirectly through a final-value check. The CESK panel is additionally smoke-tested end to end (headless Chrome/CDP) in `test/smoke-cesk.entry.ts`: that a real `println/store/kont` column renders correctly for a concrete run, that the "machine finished" status appears, and (as detailed in the companion GC note) that a real mid-run garbage collection renders and completes correctly through this exact UI path.

7 Theoretical sources

- **The CESK machine itself** — Control/Environment/Store/ Kontinuation as one configuration, and the decompose (`beginStatement/beginExpression`) / recompose (`applyKont`) discipline that realizes evaluation contexts without reifying them as data — follows Felleisen and Friedman's CEK machine [1], extended with an explicit store as in Might and Van Horn's CESK presentation [2].
- **The explicit call-stack-of-frames** (rather than one shared continuation) is, historically, the *dump* component of Landin's SECD machine [3] — the original abstract machine with explicit function call/return, predating CEK by two decades.
- **Purity enabling exact time-travel** (§4.1) is not a citation so much as a direct consequence of formalizing `step` as a mathematical function $\Sigma \rightarrow \Sigma$ rather than an in-place state mutator — the standard reason functional interpreters are easy to checkpoint, exploited here for the **Back** button rather than for, say, backtracking search.

Note on scope

This note covers the *CESK* tab (`stepperPanel.ts`) and its lockstep sibling (`comparePanel.ts`) in implementation depth. The substitution/reduction semantics behind the *Rewrite* tab (`minijavaSubstitution.ts`) is a different machine entirely (no store, no frame stack, no continuations) and is covered, both semantically and operationally, in `typing-and-evaluation-rules.tex` §4.3. The mark-and-sweep collector reachable from this same panel's **Run GC** button is covered in full in `garbage-collection.tex`.

References

- [1] M. Felleisen and D. P. Friedman. Control operators, the SECD machine, and the λ -calculus. In *Formal Description of Programming Concepts III*, 1986.
- [2] M. Might and D. Van Horn. Abstracting abstract machines. In *ICFP*, 2010.
- [3] P. J. Landin. The mechanical evaluation of expressions. *The Computer Journal*, 6(4):308–320, 1964.